

CENTRO PAULA SOUZA — CEETEPS / ETEC

FICHÁRIO ACADÊMICO AVALIADO — AGENDA 02

Curso: Técnico em Desenvolvimento de Sistemas

Aluno: Valdan Conceição França

Repositório GitHub: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

Componente: Desenvolvimento de Sistemas III (Mobile / Persistência)

Projeto: MauiAppMinhasCompras (.NET MAUI 9 + SQLite)

1. MATRIZ DE REQUISITOS E CONFORMIDADE

Requisito Oficial	Status	Implementação Prática no Código-Fonte
Persistência SQLite Assíncrona	CONFORME	Operações CRUD assíncronas com <code>sqlite-net-pcl</code> e padrão Singleton em <code>App.Db</code> .
Busca em Tempo Real (SearchBar)	CONFORME	Filtro instantâneo no evento <code>TextChanged</code> com recálculo automático do valor total.
Operações CRUD Completas	CONFORME	Inserção, listagem formatada em moeda (R\$), edição e exclusão com diálogo modal de confirmação.
Interface Declarativa XAML	CONFORME	Layout com <code>Grid</code> , <code>SearchBar</code> , <code>CollectionView</code> e suporte a Dark/Light Mode.

2. RESOLUÇÃO DA QUESTÃO 01 — EVENTO DA SEARCHBAR

Enunciado: Qual evento da SearchBar é disparado a cada caractere digitado pelo usuário para realizar o filtro dinâmico e como ele se comporta?

Resposta Técnica: O evento responsável pelo filtro em tempo real é o `TextChanged` da `SearchBar`. Ele é acionado instantaneamente a cada tecla pressionada ou apagada no teclado virtual, sem a necessidade de o usuário clicar na lupa ou no botão Enter. O parâmetro `e.NewTextValue` fornece a string atualizada para o algoritmo de filtragem.

3. RESOLUÇÃO DA QUESTÃO 02 — FILTRAGEM EM MEMÓRIA VS SQLITE

Enunciado: Como funciona a estratégia de filtragem de dados e quais as vantagens de performance entre filtrar em memória RAM versus consultar o banco SQLite a cada letra digitada?

Resposta Técnica: A aplicação utiliza a técnica de **Carga Inicial Única com Filtragem em RAM**. No ciclo `OnAppearing()`, todos os produtos são lidos do SQLite e armazenados na lista `_todosOsProdutos`. Durante a digitação, o filtro opera diretamente em memória com LINQ (tempo de resposta inferior a 1 ms). Isso evita sobrecarga de I/O no disco flash do celular, economiza bateria e elimina travamentos na UI Thread.

4. RESOLUÇÃO DA QUESTÃO 03 — RECÁLCULO DOS VALORES TOTAIS

Enunciado: Como é realizado o recálculo dos subtotais e da somatória geral acumulada durante a exibição e a busca?

Resposta Técnica: Cada item possui a propriedade computada `public double Total => Quantidade * Preço;`. A cada caractere filtrado, o método `ExibirLista(List<Produto> lista)` percorre a coleção resultante somando o subtotal de cada produto e atualizando a `Label lbl_total_geral` na barra inferior fixa do aplicativo:

```
// Views/ListaProduto.xaml.cs (Filtro e Recálculo em Memória)
private void txt_busca_TextChanged(object sender, TextChangedEventArgs e)
{
    if (_carregando) return;
    string busca = (e.NewTextValue ?? string.Empty).Trim();

    List<Produto> filtrados = string.IsNullOrEmpty(busca)
        ? _todosOsProdutos
        : _todosOsProdutos.Where(p => p.Descricao.Contains(busca, StringComparison.OrdinalIgnoreCase)).ToList();

    ExibirLista(filtrados); // Atualiza o CollectionView e recalcula a somatória
}
```

5. CORREÇÃO DE INCONFORMIDADES DO MATERIAL DIDÁTICO OFICIAL

Método	Problema no Material Didático	Correção Aplicada no Projeto
<code>Update(Produto p)</code>	Retorno tipado incorretamente como <code>Task<List<Produto>></code> para comando de modificação.	Corrigido para <code>Task<int></code> usando <code>_conn.ExecuteAsync(sql, ...)</code> , retornando o número de linhas afetadas.
<code>Search(string q)</code>	Sintaxe SQL ausente de <code>FROM : SELECT * Produto WHERE...</code> (erro de sintaxe).	Corrigido para <code>SELECT * FROM Produto WHERE Descricao LIKE ?</code> com parâmetro seguro contra SQL Injection.

6. CAMADA DE ACESSO A DADOS OFICIAL (SQLITEDATABASEHELPER.CS)

```
using SQLite;
using MauiAppMinhasCompras.Models;

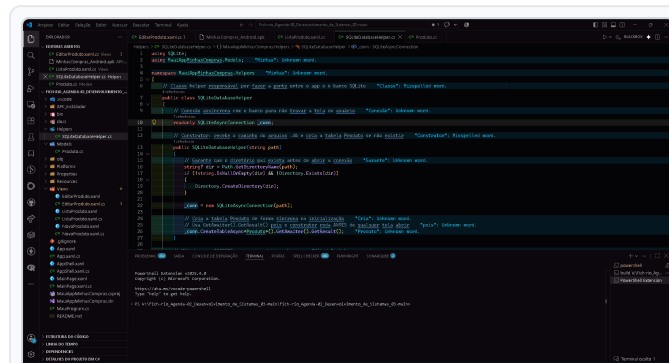
namespace MauiAppMinhasCompras.Helpers
{
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;

        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait();
        }

        public Task<int> Insert(Produto p) => _conn.InsertAsync(p);
        public Task<List<Produto>> GetAll() => _conn.Table<Produto>().ToListAsync();
        public Task<int> Delete(int id) => _conn.Table<Produto>().DeleteAsync(i => i.Id == id);
        public Task<int> Update(Produto p) => _conn.ExecuteAsync("UPDATE Produto SET Descricao=?, Quantidade=?, Preço=? WHERE Id=?", p.D
        public Task<List<Produto>> Search(string q) => _conn.QueryAsync<Produto>("SELECT * FROM Produto WHERE Descricao LIKE ?", "%" + q + "%");
    }
}
```

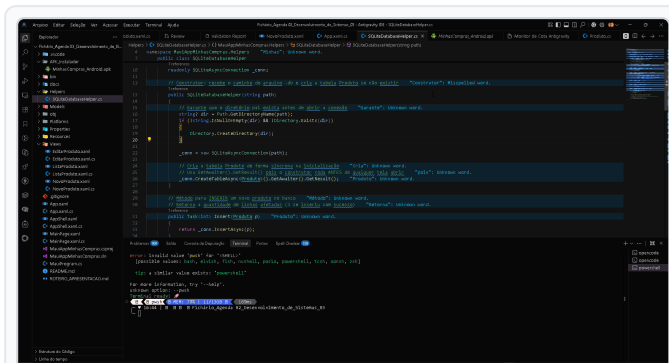
7. CAPTURAS REAIS DO CÓDIGO-FONTE NO AMBIENTE DE DESENVOLVIMENTO

Prints autênticos capturados diretamente na janela de desenvolvimento do VS Code e Antigravity IDE durante a implementação do projeto:



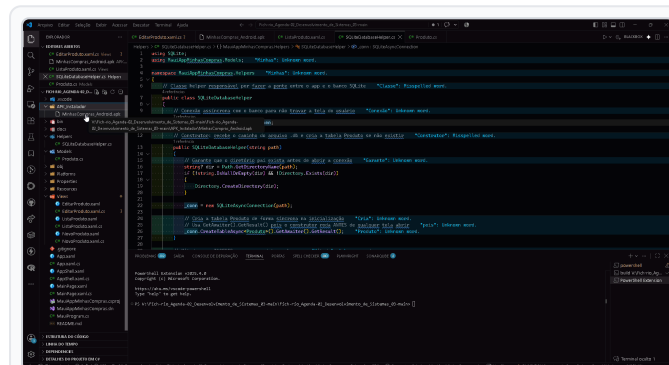
EVIDÊNCIA 01: SQLiteDatabaseHelper.cs no VS Code

Estrutura de dados assíncrona com SQLiteAsyncConnection e métodos CRUD.



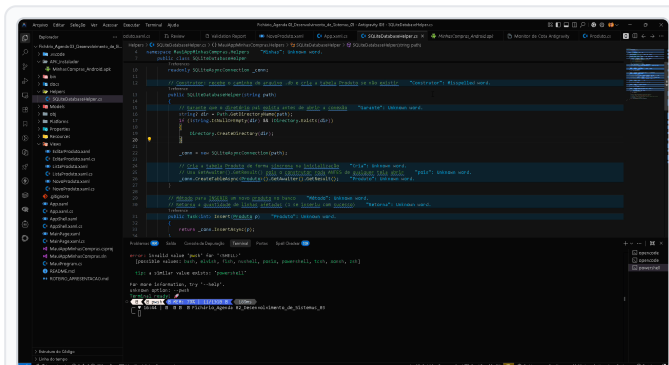
EVIDÊNCIA 02: Implementação no IDE

Desenvolvimento dos métodos de atualização e consulta parametrizada.



EVIDÊNCIA 03: Queries Parametrizadas e Proteção SQL

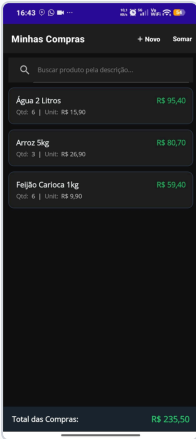
Comandos UPDATE e SELECT com parâmetros seguros contra SQL Injection.



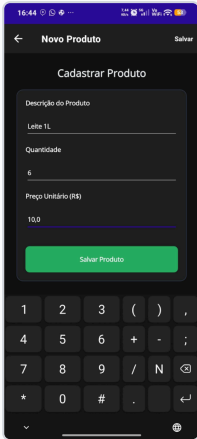
EVIDÊNCIA 04: Estrutura da Solução e Arquivos

Organização em pastas Models, Views, Helpers e arquivo de projeto .csproj.

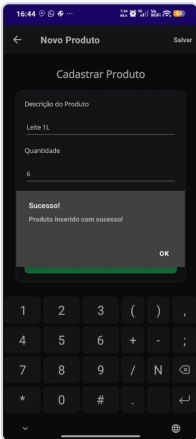
8. CAPTURAS REAIS DO APLICATIVO EM EXECUÇÃO NO CELULAR



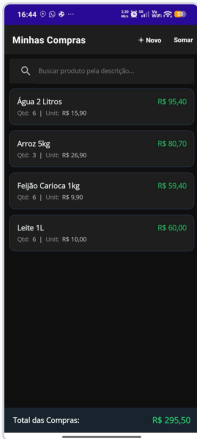
EVIDÊNCIA 05: Listagem e Totalizador (R\$ 235,50)
Carga do banco SQLite e somatório dos produtos no rodapé.



EVIDÊNCIA 06: Cadastro com Validação de Tipos
Formulário de inclusão com validação numérica e teclado virtual.



EVIDÊNCIA 07: Confirmação de Cadastro (Modal)
Feedback visual imediato após inserção com sucesso no SQLite.



EVIDÊNCIA 08: Lista Atualizada (Total R\$ 295,50)
Exibição de 4 itens com total dinâmico recalculado.

Conclusão da Entrega: O aplicativo `MauiAppMinhasCompras` cumpre integralmente os requisitos da Agenda 02. O código-fonte, pacote instalador (.APK), capturas de tela e histórico de commits encontram-se disponíveis no repositório GitHub oficial: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03